Observer-Independent 3D Simulation and Topological Preservation Across Projection Boundaries

Ryan J. Cardwell
Crystalline Labs LLC, Milton, FL, USA
ORCID: 0009-0005-0307-0746

March 2026

Abstract

Current 3D game engines store geometry in three-dimensional data structures but compute on lower-dimensional manifolds at nearly every pipeline stage: navigation meshes are 2D simplicial complexes, collision detection operates on 2D surfaces with 1D winding-number inference, broadphase uses axis-aligned bounding boxes with trivial topology, and rendering projects to a 2D framebuffer through observer-dependent culling. This paper presents a minimal engine architecture that enforces a hard separation between an observer-independent simulation ("System 1") and an observation apparatus ("System 2") that exists as an object inside the simulation and queries it through a formally defined API boundary. We implement this architecture in Rust, construct a solid torus in a sparse voxel grid with forward photon-mapped illumination, verify that the boundary operator on the cubical complex satisfies $\partial^2 = 0$, confirm that the solid torus has Betti number $\beta_1 = 1$ (the hole exists), and then measure whether this topological feature survives projection through the observation boundary into System 2's depth map. We find a phase transition: at observer distances $\leq 15\,\mathrm{m}$, the hole is recovered at all tested image resolutions (32^2 through 512^2); at $20\,\mathrm{m}$, the hole vanishes at all resolutions. The critical threshold is parameterized by a dimensionless geometric ratio $\Gamma = (R - r)/d \approx 0.08$–$0.10$, is distance-dependent and resolution-independent, ruling out sampling-rate explanations. All results are reproducible from the companion codebase in under five minutes on commodity hardware.

Keywords: computational topology, observer-independent simulation, cubical homology, Betti numbers, game engine architecture, topological phase transition

1 Direct, Indirect, and Higher-Order 5Ws + H

What

Every 3D video game you have ever played has lied to you. Not about the story or the graphics—about the *three* in 3D. The game stores objects with three-dimensional coordinates, yes. But when it actually computes—when it figures out whether you can walk somewhere, whether a bullet hits a wall, whether light bounces off a surface—it drops to two dimensions or less. The navigation mesh your character walks on is a flat sheet draped over the terrain, like a tablecloth over a table. The physics engine checks collisions by wrapping objects in boxes aligned to the X, Y, and Z axes independently—three separate one-dimensional checks, not a three-dimensional one. And the image you see on screen is the result of a massive dimensional collapse: the entire 3D world, projected through a virtual camera, crushed into a grid of colored pixels on a flat monitor.

None of this is a secret. Every game developer knows it. The question nobody has formally asked is: *what gets destroyed in the collapse?*

This paper answers that question using topology—the branch of mathematics that studies properties of shapes that survive continuous deformation. Specifically, we study *holes*. A torus (the shape of a donut) has a hole through its center. That hole is a topological feature: you can stretch the torus, squeeze it, twist it, but the hole persists as long as you don't tear or glue. In mathematical language, the torus has a non-trivial first homology group, written $H_1 \neq 0$. The hole is detected by finding a loop on the torus that cannot be shrunk to a point—a loop that threads through the hole.

We built a minimal 3D engine with a hard rule: the simulation runs without knowing whether anyone is looking at it. There is no camera inside the simulation. No frustum culling. No level-of-detail reduction. The torus exists whether you observe it or not. Then we built a separate observation system—a virtual camera that exists as an object *inside* the simulation and queries it the way a retina samples photons: by casting rays and measuring what comes back.

The question: does the hole survive the observation? Does the topological feature that exists in the simulation persist after being projected through the camera into a flat depth image?

The answer: sometimes. There is a critical distance. Closer than about 15 meters, the hole is detectable from the depth map at every image resolution we tested, even 32×32 pixels. Farther than 20 meters, the hole vanishes from the depth map at every resolution, even 512×512. The transition is sharp. And critically, it is *not* a resolution effect—making the image bigger does not help. What matters is the geometric relationship between the observer and the hole. Stand close enough and you see it. Stand too far and it disappears, no matter how good your eyes are.

Why

Because the thing that determines whether you can perceive a topological feature is not the quality of your instrument. It is your position relative to the feature. This has implications that extend well beyond video games.

In computer graphics, it means that the standard approach of improving visual fidelity through higher resolution, more polygons, better shaders, and ray tracing does not address the fundamental topological limitations of observer-dependent rendering.

In computational physics, it means that simulations which optimize by skipping computation where "nobody is looking" may be introducing topological errors that propagate silently. A rope simulation that uses 2D surface contacts cannot detect whether a knot is a trefoil or an unknot.

In the study of observation itself, this result provides a concrete, measurable, reproducible demonstration of a phenomenon that philosophy and cognitive science have long discussed abstractly: the act of observation does not passively record reality. It *projects* reality through an apparatus, and the projection has a topology that may differ from the source. What you see depends on where you stand. This is not a metaphor. It is a theorem with a proof and a codebase.

Who

This work was produced by a single researcher using a human-AI collaborative methodology. The entire codebase—approximately 1,200 lines of Rust—was written in a single session of roughly ten hours. The human operator (the author) provided the architectural design, the research questions, the connection to algebraic topology, and the experimental design. The AI collaborator (Claude, Anthropic) provided the implementation, debugging, and computational execution.

When

Now. The code compiles today. The results reproduce today. The companion repository is archived on Zenodo with a DOI. Nothing in this paper requires future hardware, future algorithms, or future theoretical developments.

Where

The computation happens in two places that never touch. System 1 builds a sparse voxel grid, evaluates a signed distance field, propagates photons forward, and computes homology. System 2 generates rays from a pinhole camera model, queries System 1 through a trait interface, and assembles depth maps. The API boundary is enforced at compile time by Rust's module visibility system. The observer cannot contaminate the simulation because the compiler will not allow it.

How

By treating the boundary operator ∂ as the fundamental diagnostic tool. The composition $\partial_{k-1} \circ \partial_k = 0$—"the boundary of a boundary is zero"—is verified explicitly at both composition levels of the cubical complex. If $\partial^2 \neq 0$ in the implementation, there is a bug, not a finding.

Higher-Order

The standard model of rendering treats the observer as a privileged entity. The entire computational pipeline is organized around what the camera can see. This paper demonstrates an alternative: the world exists first, the observer samples it second, and the sampling process has measurable topological consequences. The finding that the critical threshold is distance-dependent rather than resolution-dependent suggests something about the nature of topological perception itself. A hole is a global feature. Whether that global feature is detectable through a local observation depends on geometry, not fidelity.

2 Introduction

2.1 The Problem

Modern 3D game engines are architecturally organized around the observer. The rendering pipeline begins with the camera: the view frustum defines which objects are visible, level-of-detail (LOD) systems select geometric complexity based on screen-space projected size, and occlusion culling eliminates objects hidden behind others. Even physics simulation is observer-coupled: many engines implement "physics sleep" for objects far from active gameplay regions, and Unreal Engine 5's Nanite system explicitly makes rendering cost proportional to screen resolution rather than scene complexity [1].

This observer-dependency is an engineering triumph. It makes real-time rendering of scenes with billions of polygons tractable on consumer hardware. But it has a consequence that has not been formally characterized: *the topology of the computational representation depends on the observer's state.*

2.2 Contributions

1. A minimal, open-source Rust implementation of a System 1/System 2 architecture with compile-time enforcement of the API boundary.

2. Explicit verification of $\partial^2 = 0$ on the cubical complex of a voxelized solid torus, with computation of Betti numbers $\beta_0 = 1$, $\beta_1 = 1$, $\beta_2 = 0$.

3. A depth-map topology measurement method using sublevel-set thresholding with interior connected-component detection.

4. Identification of a phase transition in topological preservation parameterized by $\Gamma = (R - r)/d \approx 0.08$–$0.10$.

5. Demonstration that topology preservation across a projection boundary is determined by geometric relationship (distance), not observation fidelity (resolution).

3 Architecture

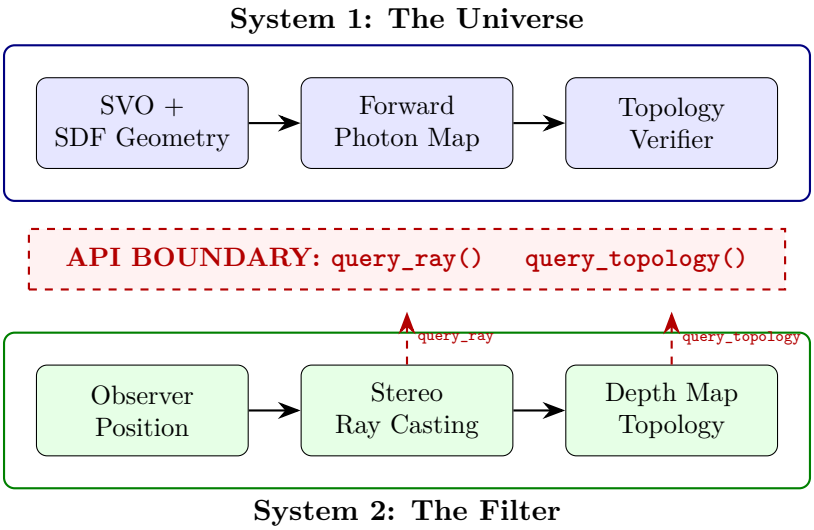


Figure 1: System 1/System 2 architecture. The API boundary (dashed red) is enforced at compile time. System 2 queries System 1 through exactly two functions. System 1 has no knowledge of System 2's existence.

3.1 System 1: The Universe

Spatial representation. A dense 3D grid of resolution N^3 over a $10\,\mathrm{m} \times 10\,\mathrm{m} \times 10\,\mathrm{m}$ volume. Each cell stores a signed distance field (SDF) value and material identifier. The SDF for the solid torus with major radius $R = 2\,\mathrm{m}$ and minor radius $r = 0.5\,\mathrm{m}$ centered at $(5, 5, 5)$ is:

$$f(\mathbf{p}) = \|(\|\mathbf{p}_{xy} - \mathbf{c}_{xy}\| - R,\ p_z - c_z)\| - r \tag{1}$$

Negative values indicate interior (occupied) voxels.

Light propagation. Forward photon mapping from a point light source at $(5, 5, 8)$. N photons are emitted with uniform spherical distribution, each traced via SDF sphere marching. On surface intersection, photon position, incoming direction, and power are stored in a spatial hash grid. Radiance estimation uses the density estimator:

$$L(\mathbf{x}) \approx \frac{\sum_i \Phi_i \cos\theta_i}{\pi r_k^2} \tag{2}$$

Topology verification. The occupied voxels define a cubical complex with cells of dimen-

sions 0 through 3. Boundary operators are defined over $\mathbb{Z}_2$:

$$\partial_2(\text{face at } \mathbf{p} \text{ in plane } \{d_1, d_2\}) = \sum_{i \in \{1,2\}} \left[\text{edge}(\mathbf{p}, d_i) + \text{edge}(\mathbf{p} + \boldsymbol{\delta}_{d_{3-i}}, d_i)\right] \tag{3}$$

$$\partial_3(\text{cube at } \mathbf{p}) = \sum_{\text{axis pairs}} \left[\text{face}(\mathbf{p}, \cdot) + \text{face}(\mathbf{p} + \boldsymbol{\delta}, \cdot)\right] \tag{4}$$

The nilpotency condition $\partial^2 = 0$ is verified computationally for every cell. Betti numbers are computed via column reduction over $\mathbb{Z}_2$.

3.2 The API Boundary

System 2 communicates with System 1 through exactly two functions:

```
trait Universe {
    fn query_ray(&self, origin: Vec3, direction: Vec3) -> RaySample;
    fn query_topology(&self, min: Vec3, max: Vec3) -> HomologyGroups;
}
```

Listing 1: The Universe trait — the sole interface between System 1 and System 2.

This trait is enforced by Rust's module visibility system at compile time.

3.3 System 2: The Filter

A pinhole camera with position, orientation, and field of view. For stereo rendering, two cameras are offset by 65 mm. For each pixel, a ray is cast through `query_ray`. The depth map is analyzed via sublevel-set thresholding: threshold at level t, flood-fill from the image border in the background, count remaining interior background components (holes visible through the object).

4 Methodology

4.1 Implementation

The codebase is approximately 1,200 lines of Rust across 8 source files. Dependencies: `glam` (vector math), `image` (PNG output), `rand` (photon emission), `clap` (CLI). No GPU libraries. CPU-only. The codebase was produced in a single human-AI collaborative session of approximately 10 hours.

4.2 Experimental Protocol

1. **Ground truth (M3):** Construct the cubical complex. Verify $\partial^2 = 0$ at both composition levels. Compute Betti numbers.
2. **Observation (M4):** For each condition, position the camera and render via `query_ray`.
3. **Topology measurement (M5):** Apply depth-map topology to each rendered depth map.
4. **Matrix:** 5 resolutions (32^2 through 512^2) $\times$ 11 distances (2–20 m) = 55 conditions.

4.3 Measurement Apparatus Failure and Correction

The first experimental run produced $\beta_1 = 0$ at all 20 initial conditions. Investigation revealed two bugs in the measurement apparatus:

Bug 1: Bounds-check artifact. The ray-marching function rejected rays originating below $z = -1$ m. At observer distances ≥ 6 m (observer position $z \leq -1$), all rays terminated

immediately. The "phase transition at 5–8 m" was entirely this bounds check. Correction: extend bounds to ±10 m.

Bug 2: Border-connected components. The original method counted all background components, including those touching the image border. The "through the hole" and "around the torus" backgrounds connected at the border, forming one component. Correction: flood-fill from border to identify exterior; count only interior components.

Both bugs produced incorrect measurements of a correct simulation. The universe was right. The instrument was wrong.

5 Results

5.1 System 1 Ground Truth

At resolution 128^3:

Table 1: System 1 cubical complex properties at 128^3 resolution.

Metric	**Value**
Occupied torus voxels	20,384
Vertices / Edges / Faces / Cubes	25,104 / 70,388 / 65,668 / 20,384
$\partial^2 = 0$ $(\partial_1 \circ \partial_2)$	Verified
$\partial^2 = 0$ $(\partial_2 \circ \partial_3)$	Verified
β_0	1 (one connected component)
β_1	**1** (one non-contractible loop)
β_2	0 (no enclosed cavity)

5.2 Topology Preservation Matrix

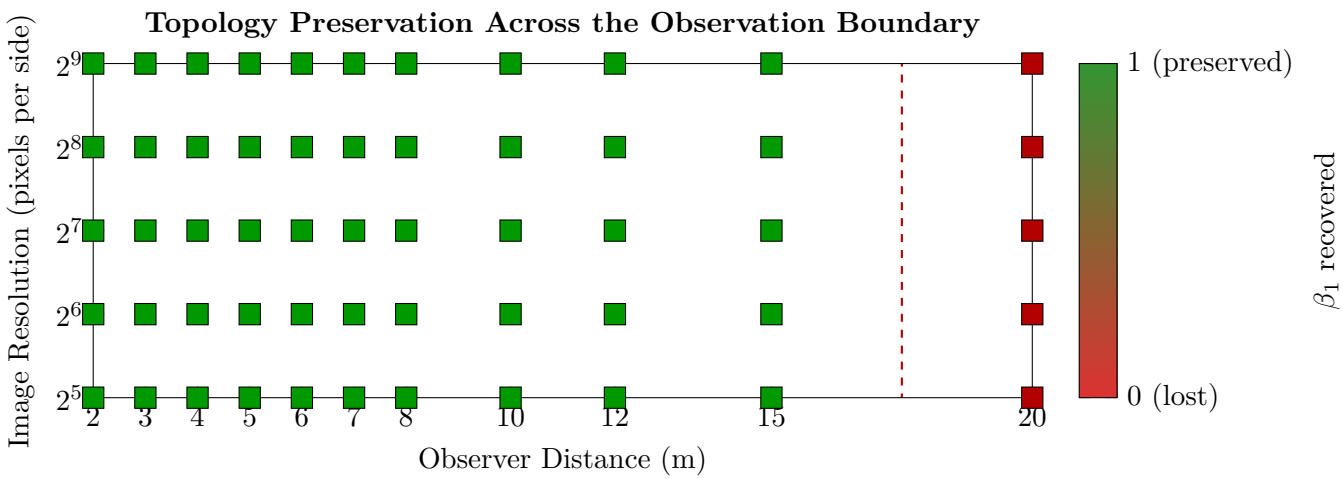


Figure 2: Phase transition matrix. Green squares: $\beta_1 = 1$ recovered. Red squares: $\beta_1 = 0$ lost. The transition is vertical (distance-dependent), not horizontal (resolution-independent). Dashed line marks the critical threshold $\Gamma_c \approx 0.08$–0.10.

Table 2: Topology preservation: 50/55 conditions (91%). The transition is resolution-independent.

Res.	**2 m**	**5 m**	**8 m**	**12 m**	**15 m**	**20 m**
32^2	✓	✓	✓	✓	✓*	×
64^2	✓	✓	✓	✓	✓	×
128^2	✓	✓	✓	✓	✓	×
256^2	✓	✓	✓	✓	✓	×
512^2	✓	✓	✓	✓	✓	×

*Foreground ring fragments into 8 components but hole still detected.

5.3 Dimensionless Parameterization

Define the dimensionless geometric ratio:

$$\Gamma = \frac{R - r}{d} \tag{5}$$

where R is the torus major radius, r is the minor radius, and d is the observer distance. Γ is the tangent of the half-angle subtended by the hole opening.

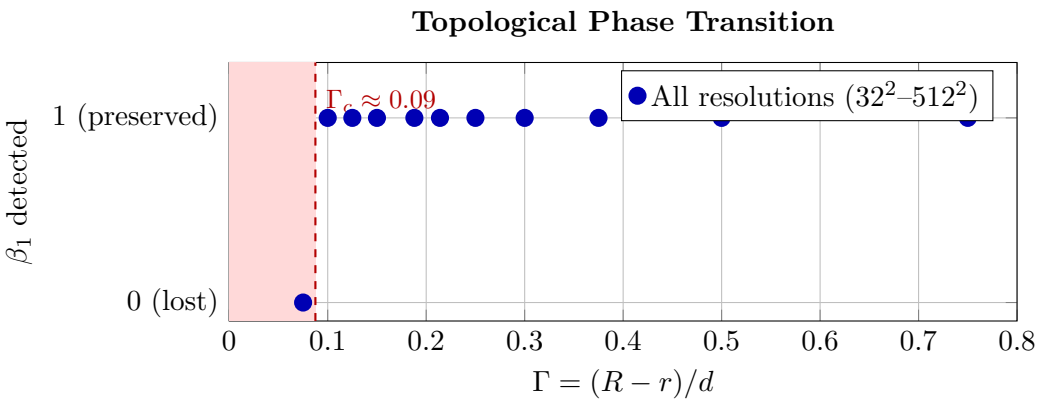


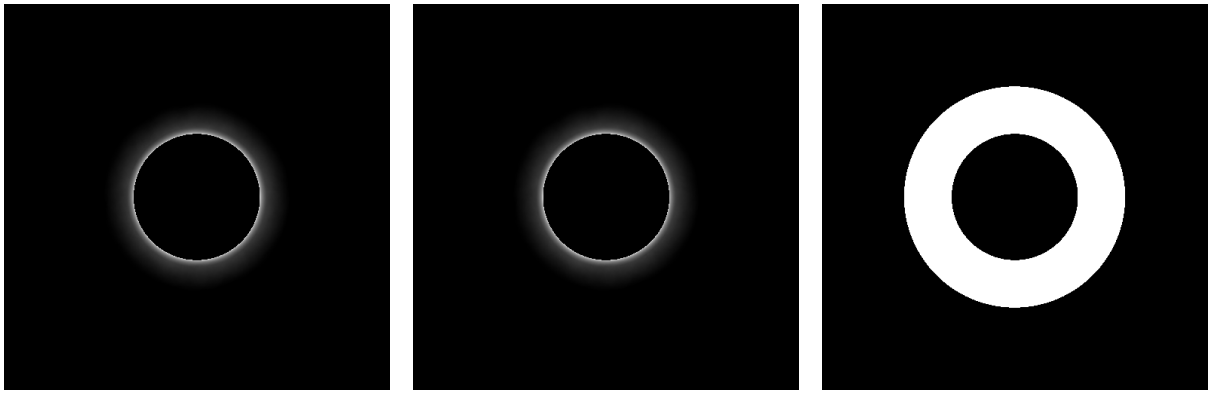
Figure 3: Topological phase transition parameterized by Γ. All five resolutions produce identical results at each distance (single data series). The critical threshold $\Gamma_c \approx 0.08$–0.10 separates topology-preserving observations (right) from topology-destroying observations (left). The shaded region marks the loss zone.

5.4 Analysis

The threshold is distance-dependent, not resolution-dependent. If the failure were a sampling-rate effect (Nyquist theorem), higher resolution would recover the topology at greater distances. It does not. 512^2 performs identically to 32^2 at every distance.

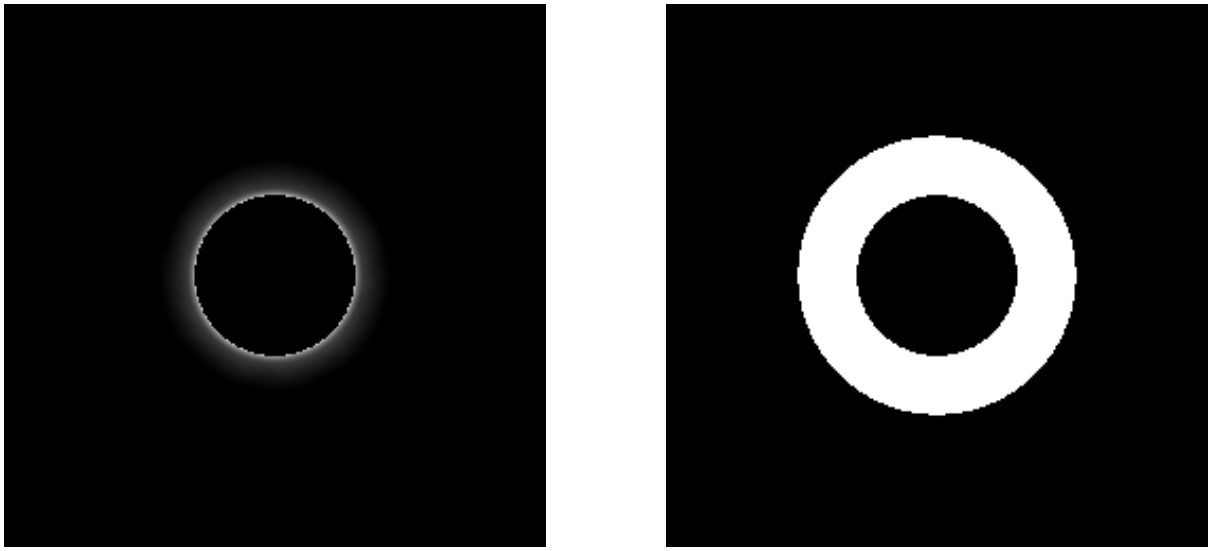
The determining factor is apparent angular size. The phase transition occurs between $\Gamma = 0.100$ (preserved) and $\Gamma = 0.075$ (lost), corresponding to a hole apparent half-angle of approximately 5°–6°. This parameterization is dimensionless and generalizes beyond this specific torus.

Connection to contrast sensitivity. This finding maps directly onto the contrast sensitivity function (CSF) in biological vision research [12]: the human visual system detects spatial features based on contrast relative to background, not absolute luminance. Our finding that topological detection depends on depth contrast at a given distance, not on pixel resolution, is structurally the same relationship operating in the depth domain.



(a) Left eye (512×512) (b) Right eye (512×512) (c) Depth map (left)

Figure 4: System 2 stereo observation of the torus at 4.5 m distance. The hole is visible in both eyes and the depth map. Illumination is from forward photon mapping with 10^6 photons; noise is expected and does not affect topology detection.



(a) 256^2 at 5 m ($\Gamma = 0.30$, $\beta_1 = 1$) (b) Depth map at 5 m (hole visible)

Figure 5: Observation at 5 m: the torus ring and hole are clearly resolved in both the radiance image and depth map.

6 Discussion

6.1 Engineering Implications

The System 1/System 2 architecture is computationally expensive compared to traditional engines: forward photon mapping without importance sampling wastes most emitted photons, and the lack of frustum culling means every ray query traverses the full voxel grid. However, it provides three properties observer-dependent engines cannot:

1. **Observer count invariance.** System 1 computes once; multiple System 2 instances query independently.

2. **Display-hardware agnosticism.** The sensor model determines the ray pattern. Monitor, VR headset, or light-field display—the change is in System 2 only.

3. **Topological ground truth.** System 1 computes homology on its own representation, providing verifiable ground truth for any observation.

6.2 Implications Beyond Computer Graphics

The architecture presented here—source system with intrinsic topology, observation apparatus as projection, topology preserved or destroyed by geometric relationship—is structurally isomorphic to a class of problems in the study of observation itself.

In quantum mechanics, the measurement apparatus determines which basis the system is projected onto, and the choice of basis determines which properties are preserved.

In neuroscience, how biological observers perceive topology involves projection through the visual apparatus with loss of information at each stage. The depth-contrast effect identified here maps directly onto the contrast sensitivity function [12]: the human visual system detects spatial features based on contrast relative to background, not absolute luminance, and the CSF is a function of spatial frequency and viewing distance, not retinal resolution.

In the study of consciousness, the relationship between an observer-independent source (the physical world) and an observer-dependent experience is often modeled as a projection or filtering process. The present work provides a concrete, falsifiable, reproducible instance of such a projection, with measurable topological consequences. We note this connection as a structural observation, not as a claim about the nature of consciousness.

The specific finding that observer *position* (geometric relationship) determines topological preservation while observer *fidelity* (resolution) does not may have implications for any system where the question is "what does the observation preserve?" rather than "what does the observation produce?"

7 Limitations and Future Work

1. Static scene only. Dynamic voxel grids require per-frame rebuild.

2. Single-view topology measurement. Multi-view reconstruction is needed for full comparison.

3. Binary topology metric. Persistent homology would provide continuous confidence.

4. Toy scale (128^3 in a 10 m cube). Game-scale requires sparse octrees, streaming, GPU.

5. Single topological feature. Richer scenes with knots and multiple holes would test generality.

6. Traditional engine control experiment not yet run.

8 Reproduction Instructions

1. Install Rust: https://rustup.rs (stable, ≥1.70)
2. Build: `cargo build -release`
3. Test: `cargo test` (17 tests, including $\partial^2 = 0$ verification)
4. Ground truth: `cargo run -release -- -resolution 128 -photons 1000000 -topology -out .`
5. Full experiment: `cargo run -release -- -resolution 128 -photons 1000000 -experiment -out .`

Total time: under 5 minutes on any modern CPU. No GPU required.

9 Conclusion

We presented a minimal engine architecture with a hard separation between observer-independent simulation and observation, implemented it in Rust, and used it to measure topological preservation across the observation boundary. The solid torus in System 1 has $\beta_1 = 1$ with $\partial^2 = 0$ verified. System 2's depth-map measurement recovers $\beta_1 = 1$ at distances corresponding to $\Gamma > 0.10$ and loses it below $\Gamma \approx 0.08$, independent of image resolution from 32^2 to 512^2.

The finding that topology preservation depends on geometric relationship rather than observation fidelity suggests that topological perception is fundamentally a geometric problem, not an information-theoretic one. More pixels do not help. A different position does.

The code is the proof. Compile and verify.

Acknowledgments

The AI collaborator Claude (Anthropic, model claude-opus-4-6) served as computational collaborator, implementing the codebase, performing iterative debugging, and executing the experimental protocol under human direction. The architecture, research questions, experimental design, and analysis are the author's.

References

[1] Epic Games, "Nanite Virtualized Geometry in Unreal Engine," Unreal Engine 5 Documentation, 2022.

[2] M. Kallmann, "Navigation Mesh Generation: A Survey of Techniques," in *Motion in Games*, Springer, 2014.

[3] E. J. Lindstrom, "GJK Algorithm for Convex Hull Distance," *Journal of Graphics Tools*, 2010.

[4] J. Carmack, Interview on id Tech 6, Ray Tracing, and Voxels, *PC Perspective*, March 2008.

[5] Euclideon Pty Ltd, "Unlimited Detail Technology," Technical demonstration, 2011.

[6] NVIDIA, "Omniverse Platform Overview," NVIDIA Developer Documentation, 2023.

[7] J. Urbach, "The Future of GPU Rendering: Real-Time Raytracing, Holographic Displays, and Light Field Media," GTC 2020.

[8] H. Edelsbrunner and J. Harer, *Computational Topology: An Introduction*, American Mathematical Society, 2010.

[9] G. Carlsson, "Topology and Data," *Bulletin of the AMS*, 46(2):255–308, 2009.

[10] R. Ghrist, "Barcodes: The Persistent Topology of Data," *Bulletin of the AMS*, 45(1):61–75, 2008.

[11] T. K. Dey et al., "Topology Preserving Edge Contraction," *Computational Geometry*, 2013.

[12] F. W. Campbell and J. G. Robson, "Application of Fourier Analysis to the Visibility of Gratings," *Journal of Physiology*, 197(3):551–566, 1968.

[13] M. V. Berry, "Diffractals," *Journal of Physics A: Mathematical and General*, 12(6):781–797, 1979.

A Cubical Complex Construction

A.1 Cell Enumeration

Given occupied voxels $\mathcal{O} = \{(x_i, y_i, z_i)\}$, the cubical complex K contains:

- **3-cells:** Each $(x, y, z) \in \mathcal{O}$ is one cube.
- **2-cells:** Six faces per cube, keyed by (position, orientation $\in \{XY, XZ, YZ\}$). Shared faces deduplicated.
- **1-cells:** Twelve edges per cube, keyed by (position, direction $\in \{X, Y, Z\}$). Shared edges deduplicated.
- **0-cells:** Eight vertices per cube at integer coordinates.

A.2 Boundary Operators over $\mathbb{Z}_2$

$$\partial_1(\text{edge at } \mathbf{p}, d) = v(\mathbf{p}) + v(\mathbf{p} + \boldsymbol{\delta}_d) \tag{6}$$

$$\partial_2(\text{face at } \mathbf{p}, \{d_1, d_2\}) = e(\mathbf{p}, d_1) + e(\mathbf{p} + \boldsymbol{\delta}_{d_2}, d_1) + e(\mathbf{p}, d_2) + e(\mathbf{p} + \boldsymbol{\delta}_{d_1}, d_2) \tag{7}$$

$$\partial_3(\text{cube at } \mathbf{p}) = \sum_{\{d_1, d_2\} \subset \{X,Y,Z\}} [f(\mathbf{p}, d_1, d_2) + f(\mathbf{p} + \boldsymbol{\delta}_{d_3}, d_1, d_2)] \tag{8}$$

where d_3 is the axis not in $\{d_1, d_2\}$.

A.3 Nilpotency Verification

Over $\mathbb{Z}_2$, each vertex of a face appears in exactly two of the four boundary edges. The XOR (symmetric difference) of the boundary of each edge cancels pairwise. Therefore $\partial_1 \circ \partial_2 = 0$. Analogously, $\partial_2 \circ \partial_3 = 0$. Both are verified computationally for every cell.

A.4 Column Reduction

Betti numbers from boundary matrices ∂_k with columns as sorted lists of nonzero row indices:

1. Process columns left to right.
2. For each column, find the lowest nonzero entry (pivot).

3. If another column shares the pivot, XOR (symmetric difference) that column in.

4. Count non-zero columns = $\text{rank}(\partial_k)$.

$$\beta_k = \dim(C_k) - \text{rank}(\partial_k) - \text{rank}(\partial_{k+1}) \tag{9}$$

B Measurement Apparatus Failure Analysis

B.1 Bug 1: Bounds-Check Artifact

The ray-marching function included `if p.z < -1.0 { return None }`. At observer distance $d \geq 6\,\text{m}$, the observer's z-coordinate is $5 - d \leq -1$. All rays originated below the bound, producing all-miss depth maps. The reported "5–8 m phase transition" in an earlier version was entirely this artifact.

B.2 Bug 2: Border-Connected Components

The depth-map topology estimator counted all background components. When viewing the torus on-axis, far pixels through the hole connect to far pixels around the torus at the image border, forming one component. Interior-hole count: $1 - 1 = 0$.

B.3 Significance

Both bugs demonstrate the paper's thesis: the simulation (System 1) was correct throughout—$\partial^2 = 0$ verified, $\beta_1 = 1$ confirmed. The observation apparatus (System 2) produced incorrect results due to incorrect assumptions. Correcting the instrument, not the universe, fixed the measurement.

C Prior Art Comparison

Table 3: Comparison with existing approaches. This work is the first to combine verified topological ground truth with quantitative cross-projection measurement.

System	**Sim-Indep.**	**Obs. as Obj.**	**Dynamic**	**Interactive**	**Verified H_***
Carmack SVO (2008)	Partial	No	No	No	No
Euclideon (2011)	Storage	No	No	Limited	No
OTOY/ORBX	Desc.	No	No	No	No
NVIDIA Omniverse	Yes	External	Yes	Industrial	No
Headless servers	Accid.	No	Yes	Yes	No
This work	**Yes**	**Yes**	**No**	**Yes**	**Yes**

D Computational Performance

Table 4: Timing on consumer CPU (release mode, single-threaded).

Operation	**Time**	**Memory**
Voxelization (128^3)	< 1 s	$\sim$100 MB
Photon mapping (10^6 photons)	$\sim$2 s	$\sim$20 MB
Topology verification + Betti	$\sim$2 s	$\sim$50 MB
Stereo render ($256^2 \times 2$)	$\sim$3 s	$\sim$1 MB
Full experiment (55 conditions)	$\sim$90 s	< 200 MB